Likelihood-Based Attribute Retrieval with a Fine-Tuned CLIP ViT

UPAR Challenge 2027, Track 2: Attribute-Based Person Retrieval (RWS Workshop @ WACV 2027)

Arsh Al Aman (23K-0078) · Rao Ghulam Mohiuddin (23K-0001) · Sheikh Naveed Azeemi (23K-0003)
FAST National University of Computer and Emerging Sciences · Computer Vision · Instructor: Mr. Syed Bilal Ahsan · Codabench team: arshalaman

Abstract. We take part in Track 2 of the UPAR Challenge 2027, which asks for a ranking of pedestrian images given a textual-style description of 40 binary attributes, evaluated on a hidden surveillance domain. We fine-tune the visual encoder of CLIP ViT-B/16 as an attribute recogniser and replace the starter kit's L1 distance with a likelihood-based score, an agreement term for near misses and a query-side normalisation. On the validation split this raises mADM from 0.113 (organiser baseline) to 0.746; the scoring alone adds +0.053 mADM over L1 with the same model. On the hidden test set our final two-model ensemble reaches **0.624 mADM (rank 32)**. We analyse why the test score is lower, show that the ranking is robust to simulated domain shift, and identify early stopping, rare attributes and the attribute model itself as the main levers for future work.

1. Introduction and Task

1.1 Choice of track

The UPAR Challenge 2027 offers three tracks on the unified UPAR benchmark: (1) pedestrian attribute recognition (PAR), (2) attribute-based person retrieval (ABPR) and (3) 2D human pose estimation. We chose **Track 2**. It builds directly on a strong attribute recogniser, so a single model serves both PAR and retrieval (we also report Track 1 metrics), and its rules allow general foundation models such as CLIP, whereas Track 3 forbids them. The retrieval step also leaves room for a contribution of our own beyond the backbone: how to turn attribute probabilities into a ranking.

1.2 Problem definition

A query is not an image but a full description: a vector $q \in \{0,1\}^{40}$ over age (young/adult/old), gender, hair length, upper- and lower-body clothing length, colour and type, and accessories (backpack, bag, glasses, sunglasses, hat). For every query the system ranks a gallery of pedestrian crops so that images whose 40 attributes all equal the query come first. The gallery comes from a hidden surveillance domain that is never seen during training, so the central difficulty is *generalisation under domain shift*. Submissions are code: our run.py is executed on an NVIDIA A10 (24 GB) with no network access, only torch, torchvision, numpy and Pillow, and a 5-minute limit.

1.3 Data

Table 1. Training and validation data (UPAR harmonised annotations).

Split	Market-1501	PA-100K	PETA	Total images	Queries
Train	10,000	79,001	8,668	97,669	7,204
Validation	16,458	9,986	6,963	33,407	3,462

All labels are binary; age is (almost) always one-hot and upper-body colour is single-valued in 98.8% of images. Attribute frequencies are highly imbalanced: positive rates range from 0.2% (orange trousers) to 93.5%. A query in the validation split has on average 9.6 matching images (median 2).

1.4 Evaluation metric

Let $r_1 < \ldots < r_R$ be the ranks of the R exact matches of a query. Besides mAP, Rank-k and mINP, the ranking metric is the **mean Average Degree of Match (mADM)**. The degree of match dom(g) is the share of the 40 attributes on which image g agrees with the query; it is normalised per query so that an average gallery image earns 0 and a perfect match 1: ndom(g) = max(0, (dom(g) − mean dom)/(max dom − mean dom)). Then

$$ADM = (1/R)\ \Sigma_k\ [\ ndom(g_1) + \ldots + ndom(g_{rk})\ ]\ /\ r_k$$

so, unlike AP, an image that differs from the query in one attribute is far better than one that differs in twenty. We re-implemented all metrics and checked them against a literal implementation of these formulas (identical to six decimals).

1.5 Baselines

The organisers' example submission predicts the training prior (the share of positives per attribute) for every image and ranks by the L1 distance $\Sigma_k |q_k - p_k|$. Because it ignores the pixels, every image gets the same score; it reaches 0.113 mADM on validation. Previous UPAR editions established the standard two-step recipe: a PAR network predicts attribute probabilities, and queries are ranked by a distance to them. We use the same recipe with our model and L1 distance as a strong internal baseline (0.694 mADM), and improve both steps.

2. Method

2.1 Attribute recogniser

The backbone is the visual tower of OpenAI **CLIP ViT-B/16** [3], pretrained on web-scale image–text pairs, which helps on unseen cameras. Pedestrian crops are resized to 256×128, and the 14×14 positional embedding is bicubically resampled to the 16×8 patch grid. The head concatenates the class token with the mean of the 128 patch tokens (global plus part evidence) and applies LayerNorm, dropout 0.1 and a linear layer with 40 logits. Because the evaluation container offers no open_clip, we re-implemented the ViT in plain PyTorch with CLIP-compatible parameter names; on random inputs it reproduces open_clip's outputs exactly (maximum difference 0.0).

2.2 Training

Table 2. Training configuration (identical for all runs except the epoch count and the data split).

Setting	Value
Loss	binary cross-entropy, unweighted (keeps probabilities calibrated)
Optimiser	AdamW, weight decay 0.05 (none on norms, biases, embeddings)
Learning rate	6e-5 last CLIP block, layer-wise decay 0.75 per block; head 1e-3
Schedule	0.5 epoch linear warm-up, cosine decay to 0
Batch / precision	128 / fp16 mixed precision, gradient clipping at 1.0
Weight averaging	EMA of the weights, decay 0.9995 (the EMA model is evaluated)
Augmentation	flip; scale 0.85–1.1 and shift ±8%; low-resolution simulation (p 0.3, down to 25–60%); brightness/contrast ±25%, saturation ±20%; random erasing (p 0.3, 2–12% area); no hue jitter
Input pipeline	images pre-resized once into a 1 GB JPEG cache; augmentation on the GPU
Epochs	10 for the development run, 3 for all submitted models
Hardware	one free Colab NVIDIA T4; ~12 min/epoch (train), ~14.5 min/epoch (train+val)

Two choices are specific to surveillance footage. The *low-resolution simulation* (down-sampling and up-sampling a random 30% of each batch) imitates distant cameras. We use *no hue jitter* at all, because colour is itself one of the labels: changing it would corrupt the target.

2.3 Retrieval scoring

L1 distance treats a confident mistake like an uncertain one: an image with p = 0.99 for "backpack" is penalised only slightly for a query without a backpack. We instead score by the log-likelihood that the image shows exactly the query, assuming independent attributes, plus an agreement term that rewards near misses, which is what mADM measures:

$$s(q,g) = \Sigma_k \left[q_k \log \hat{p}_k + (1-q_k) \log(1-\hat{p}_k) \right] + \alpha \Sigma_k \left[q_k p_k + (1-q_k)(1-p_k) \right]$$

where $\hat{p}$ is p clipped to $[\varepsilon, 1-\varepsilon]$ so that a single extreme prediction cannot dominate. The second term is the expected number of agreeing attributes, i.e. 40 minus the L1 distance. By construction of the challenge, each gallery image matches exactly one query, so we add a *query-side normalisation* similar to dual-softmax re-ranking in cross-modal retrieval:

$$s'(q,g) = s(q,g) + \beta \cdot \log \mathrm{softmax}_q(s(\cdot,g) / \tau)$$

This lowers images that fit many different queries equally well. It is fully automatic and uses no knowledge of the test set, as the rules require. α = 1, β = 2, τ = 10 and ε = 1e-3 were chosen by a staged grid search on the validation split (Table 4). Scores for all queries and images are computed as two matrix products on the GPU in chunks, so no Q×G×40 tensor is ever built.

2.4 Inference and submission

run.py loads every model*.pt file in the zip, reads and resizes images with a 16-thread pool while the GPU processes the previous batch, runs the model(s) in fp16 on each image and its mirror image, averages the probabilities, and returns a Q×G similarity matrix. A single-model zip is 172 MB; the final two-model ensemble is 345 MB. Before every upload the zip was unpacked and executed exactly as the ingestion program calls it, including an unreadable image, which is replaced by a blank crop instead of crashing the run.

3. Experiments and Results

Unless stated otherwise, all 33,407 validation images form one gallery that is ranked for all 3,462 validation queries, using the development model (trained on the training split only).

3.1 Training dynamics and model selection

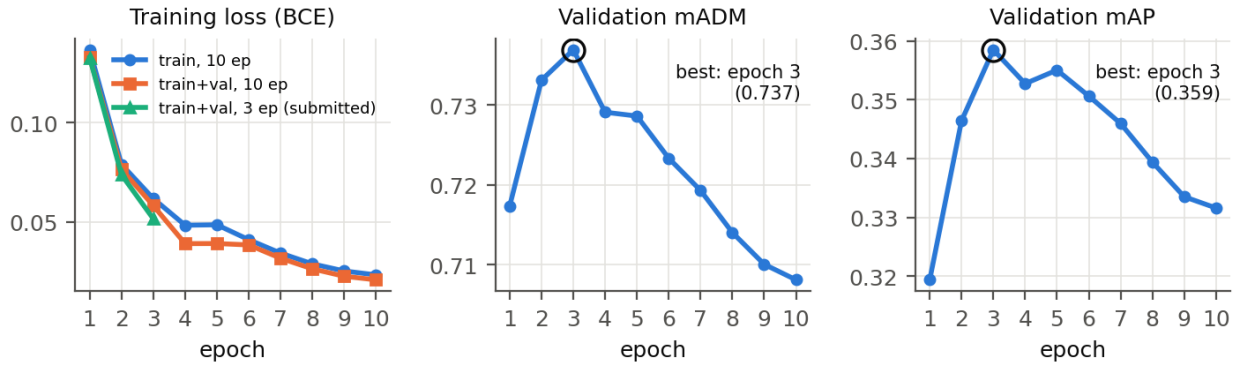


Figure 1. Training loss and validation retrieval metrics per epoch. Validation mADM peaks after 3 epochs and then declines while the training loss keeps falling. Orange: train+val for 10 epochs (not submitted). Green: train+val for 3 epochs.

The model converges very quickly: validation mADM peaks at **0.737 after 3 epochs** and then decreases steadily to 0.708 at epoch 10 (mAP: 0.359 → 0.332), although the training loss keeps falling (0.062 → 0.023). CLIP's pretrained features generalise well, and longer fine-tuning over-specialises them to the three source datasets, which is exactly what we must avoid under domain shift. We therefore fixed the training length at 3 epochs for every submitted model and discarded a 10-epoch train+val run we had already trained. Following the rules, we also trained on train and validation together (131,076 images) for 3 epochs; this model cannot be validated, as it has seen the validation images.

3.2 Scoring ablation

Table 3. Ablation of the retrieval scoring on the validation split (same attribute model, best epoch).

Method	mADM	mAP	R-1	R-5	R-10	mINP
Organizer baseline (attribute prior + L1)	0.113	0.001	0.000	0.000	0.001	0.001
CLIP model + L1 distance	0.694	0.233	0.279	0.420	0.477	0.125
+ log-likelihood scoring	0.735	0.360	0.365	0.592	0.681	0.242
+ agreement term + query normalisation (ours)	**0.746**	**0.340**	**0.351**	**0.562**	**0.639**	**0.221**

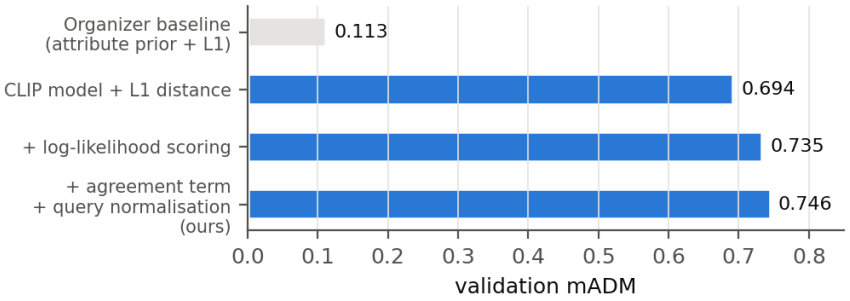


Figure 2. Validation mADM for the scoring variants of Table 3.

Replacing the organisers' attribute prior with our fine-tuned CLIP raises validation mADM from 0.113 to 0.694. Swapping L1 for likelihood scoring alone adds +0.041 mADM and improves mAP by 55% relative (0.233 → 0.360) at no extra cost. Adding the agreement term and the query normalisation brings mADM to **0.746** (+0.053 over L1). These two terms trade a little exact-match precision (mAP) for a better ordering of near misses, which is exactly what mADM rewards.

3.3 Sensitivity of the scoring parameters

Table 4. Selected points of the validation grid search.

Scoring	mADM	mAP	Rank-1
L1 distance (starter kit)	0.694	0.233	0.279
LL, ε = 1e-4	0.735	0.360	0.365
LL, ε = 1e-3	0.737	0.358	0.364
LL, ε = 1e-2	0.735	0.343	0.358
LL, ε = 3e-2	0.728	0.322	0.341
LL + α = 1, ε = 1e-3	0.744	0.348	0.357
LL + α = 3, ε = 1e-3	0.742	0.324	0.342
LL + α = 10, ε = 1e-3	0.727	0.284	0.317
+ query norm β = 1, τ = 10	0.746	0.343	0.354
+ query norm β = 2, τ = 10 (chosen)	**0.746**	**0.340**	**0.351**
+ query norm β = 4, τ = 10	0.745	0.330	0.345

Clipping matters: a too-large ε flattens the likelihood (mADM 0.737 → 0.728 from ε = 1e-3 to 3e-2). A small agreement weight helps (α = 1), but a large one turns the score back into L1 and loses both mADM and mAP. The query normalisation adds about +0.002 mADM, and every combination of β between 0.5 and 4 with τ between 3 and 20 lies within 0.002 of the best setting, so the choice is stable rather than tuned to noise.

3.4 Results per source dataset

Table 5. Per-source validation results (each source's images as its own gallery, chosen scoring).

Source dataset	Images	Queries	mADM	mAP	Rank-1
Market1501	16458	956	0.751	0.348	0.400
PA100k	9986	1479	0.721	0.342	0.346
PETA	6963	1763	0.790	0.463	0.429

PETA is the easiest source (mADM 0.790) and PA-100K the hardest (0.721), even though PA-100K dominates the training set: its images are small, low-resolution surveillance crops, closer to the conditions we expect on the hidden test set.

3.5 Per-attribute analysis

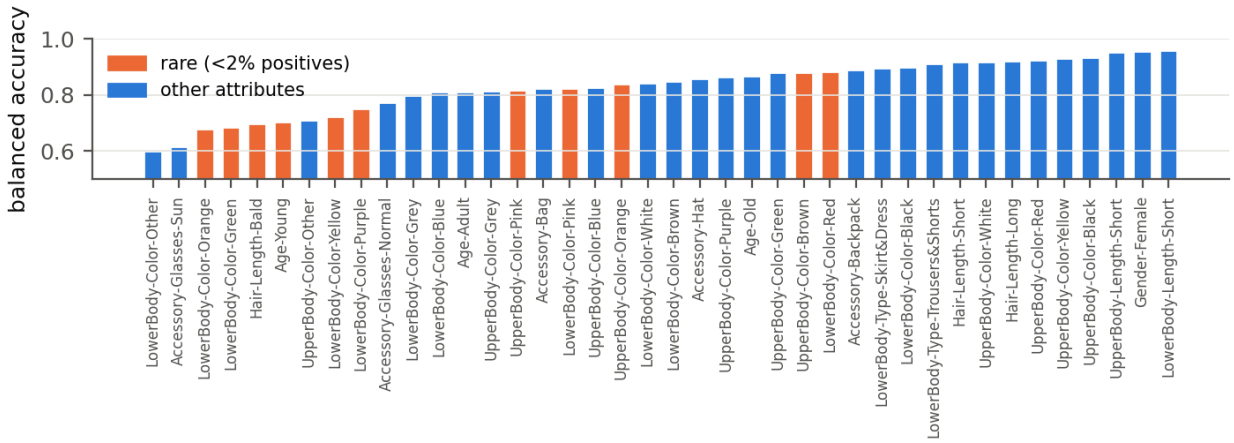


Figure 3. Validation balanced accuracy per attribute (threshold 0.5). Rare attributes (orange, <2% positives) and the catch-all "other" colours are the weakest.

Table 6. Hardest and easiest attributes on validation.

	Attribute	Balanced accuracy	Positive rate
hardest	LowerBody-Color-Other	0.602	3.7%
hardest	Accessory-Glasses-Sun	0.615	2.2%
hardest	LowerBody-Color-Orange	0.678	0.2%
hardest	LowerBody-Color-Green	0.686	1.5%
hardest	Hair-Length-Bald	0.697	1.3%
hardest	Age-Young	0.703	1.1%
easiest	LowerBody-Length-Short	0.958	37.5%
easiest	Gender-Female	0.957	42.9%
easiest	UpperBody-Length-Short	0.953	62.9%
easiest	UpperBody-Color-Black	0.933	25.6%

Global, well-represented attributes (gender, clothing length) exceed 95% balanced accuracy. The weakest are rare colours (orange, green and purple trousers), bald and young people, the ambiguous catch-all "other" colour, and sunglasses, which are only a few pixels wide in low-resolution crops. Because a query is matched only when all 40 attributes agree, these rare errors dominate the retrieval loss.

3.6 Track 1 metrics as a by-product

Table 7. Pedestrian attribute recognition metrics of the same model on validation (threshold 0.5).

mA	Instance accuracy	Instance F1	Track 1 challenge average
0.831	0.823	0.893	0.861

3.7 Robustness of the scoring to domain shift

To test whether our scoring could be the cause of a lower test score, we degraded the validation probabilities in ways a domain shift might: random logit noise, noise plus over-confidence, and a random per-attribute shift of the prior. Each setting was evaluated on 1,200 random validation queries.

Table 8. Validation mADM under simulated shift of the predicted probabilities.

Simulated shift	L1	Log-likelihood	Ours
clean	0.695	0.735	0.746
noise (std 1)	0.670	0.710	0.722
noise (std 2) + over-confident	0.658	0.685	0.697
attribute prior shift	0.674	0.725	0.734

Our scoring stays ahead of L1 by 0.04–0.06 mADM in every case, so the ranking step is robust and is not the bottleneck.

3.8 Hidden test set

Table 9. Hidden test set (Codabench, final leaderboard). Our final submission is the ensemble; final rank 32.

Submission	mADM	mAP	R-1	R-5	R-10	mINP
Leading entry (rank 1)	0.699	0.416	0.397	0.560	0.632	0.292
Ours: train only, 3 epochs	0.619	0.348	0.378	0.571	0.653	0.250
Ours: train+val, 3 epochs	0.613	0.339	0.359	0.564	0.652	0.241
Ours: ensemble of both (final)	**0.624**	**0.353**	**0.385**	**0.575**	**0.658**	**0.252**

Our final ensemble reaches **mADM 0.624** (rank 32), against 0.699 for the winner. Our Rank-5 and Rank-10 are higher than the winner's, while mADM, mAP and mINP are lower. Three observations stand out. (i) Our test mAP is as high as on validation (0.353 vs. 0.340) while mADM falls from 0.746, so on the new cameras the model mainly orders near-miss images worse; together with Table 8 this points to the attribute model, not the ranking, as the bottleneck. (ii) Adding the validation images did not help on its own (0.613 vs. 0.619): they come from the same three source datasets, and a full learning-rate decay within 3 epochs fits them more tightly than the 3-epoch checkpoint of a 10-epoch schedule, the same effect as in Fig. 1. (iii) Averaging the two models gives the best result on every metric, as expected from an ensemble under domain shift. Screenshots of the Codabench submissions and leaderboard are attached as proof of submission.

4. Challenges

- **Domain shift and overfitting.** The test cameras are unseen and the validation split comes from the training datasets, so validation overestimates test performance (0.746 vs. 0.619). Validation mADM falls after 3 epochs, so the number of epochs turned out to be the most important regularisation setting.
- **Rare and fine-grained attributes.** Positive rates down to 0.2% and tiny accessories at low resolution (Table 6). A single wrong attribute breaks an exact match.
- **A hidden 4.5× slowdown.** PyTorch reports bfloat16 as supported on the T4, but it is emulated there: training ran at ≈30 img/s instead of 139 img/s with fp16 (53 min instead of 12 min per epoch). We found it from the throughput logs and fixed it by checking the GPU's compute capability.
- **Free-GPU quotas.** Colab ended sessions after a few hours of GPU use. Training therefore saves a full checkpoint (model, EMA, optimiser, scaler) to Google Drive after every epoch and resumes automatically, which let us continue on other accounts.
- **Offline code submission.** No network, only plain PyTorch and a 5-minute limit. We re-implemented CLIP without external libraries, verified it numerically and tested each zip exactly as the ingestion program calls it.
- **Submission logistics.** Near the deadline the Codabench queue took many hours per submission, and only the latest submission counts on the leaderboard. We planned uploads accordingly: a first model as early proof, then the alternatives, and the ensemble last.
- **No validation for the final data split.** A model trained on train+val cannot be validated, so its training length had to be transferred from the development run.

5. Lessons Learned

- With a strong pretrained backbone, **less fine-tuning generalises better** to an unseen domain; validation curves must be read with the domain gap in mind.
- The step from probabilities to a ranking deserves as much attention as the model: a principled likelihood score was worth +0.04–0.05 mADM at zero cost.
- More data from the same source domains is not a substitute for diversity; ensembling was the most reliable test-time gain.

- Engineering details (numeric formats, resumable training, offline packaging) decide whether an approach can be run at all on free hardware.

6. Possible Improvements

- **Larger and more diverse ensembles:** several seeds, or ViT-B with ViT-L/14, within the 5-minute budget. Averaging probabilities improves calibration, which the likelihood scoring relies on.
- **Structured outputs:** softmax heads for mutually exclusive groups (age, clothing colours) and a learned correction for attribute dependencies instead of assuming independence.
- **Rare attributes:** class-balanced or focal loss followed by re-calibration (temperature scaling), and higher-resolution or part-based features for small accessories.
- **Domain generalisation:** stronger style and camera augmentation, frozen early layers or parameter-efficient fine-tuning (e.g. adapters) that keep CLIP's features closer to pretraining.
- **Retrieval-aware training:** a listwise loss on query–image scores, and stronger allowed backbones (SigLIP, EVA-CLIP).

7. Code and Reproducibility

All code, training logs, validation results and figures are in our repository: https://github.com/arsh608/UPAR-Challenge-Track-2. The complete pipeline (download, cache, training, evaluation, packaging) runs in a single Colab notebook on a free T4, and every table and figure of Section 3 except the test results can be regenerated on a CPU from the saved validation probabilities with analysis.py.

References

[1] A. Specker, M. Cormier, J. Beyerer. UPAR: Unified Pedestrian Attribute Recognition and Person Retrieval. WACV 2023.

[2] M. Cormier, A. Specker, et al. UPAR Challenge 2024: Pedestrian Attribute Recognition and Attribute-Based Person Retrieval – Dataset, Design, and Results. WACV Workshops 2024.

[3] A. Radford et al. Learning Transferable Visual Models From Natural Language Supervision (CLIP). ICML 2021.

[4] A. Specker, J. Beyerer. Balanced Pedestrian Attribute Recognition for Improved Attribute-based Person Retrieval. 2023.

[5] X. Liu et al. HydraPlus-Net (PA-100K). ICCV 2017. Y. Deng et al. Pedestrian Attribute Recognition at Far Distance (PETA). ACM MM 2014. L. Zheng et al. Scalable Person Re-identification: A Benchmark (Market-1501). ICCV 2015.